



دانشگاه هرمزگان
دانشکده فنی و مهندسی
گروه مهندسی برق و کامپیوتر

داده‌ساختارها و الگوریتم‌ها

جلسه چهارم – تحلیل الگوریتم‌های ترتیبی و مقدمه‌ای بر روش تقسیم و حل

دکتر شهرام گلزاری

پاییز ۱۴۰۵

در پایان این جلسه انتظار داریم که:

- بتوانیم زمان اجرای یک الگوریتم ترتیبی را با قاعده‌های صفحه ۸۰ کتاب دکتر قدسی به صورت مجانبی به دست آوریم.
- با الگوریتم‌های بازگشتی و بیان زمان اجرای آن‌ها با **رابطه‌ی بازگشتی** آشنا شویم.
- الگوی کلی روش **تقسیم و حل** را بشناسیم و آن را در مرتب‌سازی ادغامی ببینیم.

تحليل الگوریتم‌های ترتیبی

«چنانچه نشان داده‌ایم، الگوریتم‌ها به دو دسته‌ی اصلی تقسیم می‌شوند: ترتیبی و بازگشتی. الگوریتم‌هایی را که بازگشتی نیستند ترتیبی می‌گوییم. در این بخش با روش تحلیل هر کدام از این‌ها آشنا می‌شویم. الگوریتم‌های ترتیبی را معمولاً با شمارش تعداد دفعات اجرای دستوری که برحسب اندازه‌ی داده‌ی ورودی، بیش‌ترین بار اجرا می‌شود تحلیل می‌کنیم. اما زمان اجرا و میزان حافظه‌ی مصرفی الگوریتم‌های بازگشتی با رابطه‌های بازگشتی بیان می‌شوند.»

نوع الگوریتم	ابزار تحلیل	مثال
ترتیبی (sequential)	شمارش اجرای پرتکرارترین دستور	مرتب‌سازی حبابی
بازگشتی (recursive)	رابطه‌ی بازگشتی	برج‌های هانوی، ادغامی

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۷۹

«برای به‌دست آوردن زمان اجرای یک الگوریتم ترتیبی فرض می‌کنیم که اجرای هر «دستور ساده» به زمان ثابتی نیاز دارد. بنابراین اگر تعداد ثابتی دستور ساده پشت سر هم باشند، در مجموع زمانی ثابت می‌گیرند.»

«با فرض ثابت بودن k ، اگر T خود شامل k تکه‌برنامه با زمان‌های اجرای $T_1(n) = O(f_1(n))$ ، $T_2(n) = O(f_2(n))$ تا $T_k(n) = O(f_k(n))$ باشد، در آن صورت

$$T(n) = \sum_{i=1}^k T_i(n) = O\left(\max_{1 \leq i \leq k} (f_i(n))\right)$$

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۸۰

«اگر T ساختار حلقه داشته باشد (مانند `while`، `repeat`، `for` و امثال آن) و $g(n)$ بار تکه‌برنامه‌ای دیگر با زمان اجرای $S(n) = O(f(n))$ را تکرار کند،

$$T(n) = g(n)S(n) = O(g(n)f(n))$$

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۸۰

«اگر T ساختار if باشد و بخش‌های then و else آن به ترتیب زمان‌های $T_1(n) = O(f_1(n))$ و $T_2(n) = O(f_2(n))$ را داشته باشند،

$$T(n) = O(\max\{f_1(n), f_2(n)\}).$$

البته معمولاً $T(n) = \max\{T_1(n), T_2(n)\}$ ولی ممکن است یکی از قسمت‌های then یا else هیچ‌گاه اتفاق نیفتد.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۸۰

«به‌طور شهودی، معمولاً مرتبه‌ی زمان اجرای یک الگوریتم، همان مرتبه‌ی تکه‌ای از برنامه است که بیش‌ترین زمان را می‌گیرد، چرا که همیشه برای تابع رشد، بدترین حالت مد نظر است.»

صفحه ۸۰

«البته این روش، کران بالای مقدار مورد تحلیل را به‌دست می‌آورد که آن‌را با نماد O نشان داده‌ایم. گاهی برای به‌دست آوردن پیچیدگی دقیق یا Θ باید از روش‌های پیچیده‌تری مثل «روش سرشکنی» استفاده کرد.»

صفحه ۸۱

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی

Exercise: Find the Order of the Running Time

(a)

```
int x = 0;
for (int i = 0; i < n; i++) {
    if (i % 5 == 0) {
        for (int j = 0; j < n; j++) {
            if (i == j) {
                x += i * j;
            }
        }
    }
}
```

(b)

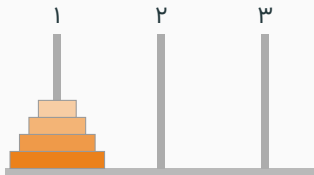
```
int x = 0;
for (int i = 0; i < n; i++) {
    if (n < 100000) {
        for (int j = 0; j < n; j++) {
            x += 1;
        }
    } else {
        x += 1;
    }
}
```

(c)

```
int x = 0;
if (n % 2 == 0) {
    for (int i = 0; i < n * n * n * n; i++) {
        x++;
    }
} else {
    for (int i = 0; i < n * n * n; i++) {
        x++;
    }
}
```

دانشگاه واشنگتن، CSE 373 23SP، قسمت دوم – Link

الگوریتم‌های بازگشتی: برج‌های هانوی



مثالی از مسئله‌ی برج‌های هانوی

«در این مسئله‌ی قدیمی و آشنا، n سکه‌ی سوراخ‌دار با قطرهای متفاوت به‌ترتیب قطرشان در میله‌ی شماره‌ی ۱ از ۳ میله‌ی موجود قرار دارند (سکه‌ی بزرگ‌تر در پایین سکه‌هاست). شکل ۳-۷ مثالی از حالت اولیه‌ی این مسئله را با ۴ سکه نشان می‌دهد. هدف، انتقال همه‌ی سکه‌ها به میله‌ی شماره‌ی ۳ به‌کمک میله‌ی شماره‌ی ۲ است به‌طوری‌که،

- هر بار فقط یک سکه جابه‌جا شود،
- در هر حرکت سکه‌ی کوچک‌تر بر روی سکه‌ی بزرگ‌تر قرار گیرد، و
- تعداد حرکت‌ها کمینه باشد.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه‌های ۸۴ و ۸۵ (بازترسیم شکل ۳-۷)

برای جابه‌جا کردن بزرگ‌ترین سکه، باید همه‌ی $n - 1$ سکه‌ی دیگر روی میله‌ی ۲ باشند. پس:



۱. $n - 1$ سکه‌ی بالایی را از میله‌ی ۱ به میله‌ی ۲ ببر .
۲. بزرگ‌ترین سکه را از میله‌ی ۱ به میله‌ی ۳ ببر .
۳. $n - 1$ سکه را از میله‌ی ۲ به میله‌ی ۳ ببر .

 $\text{HANOI}(n, \text{src}, \text{dst}, \text{tmp})$

```

1 if  $n > 0$  then
2    $\text{HANOI}(n - 1, \text{src}, \text{tmp}, \text{dst})$ 
3   move disk  $n$  from  $\text{src}$  to  $\text{dst}$ 
4    $\text{HANOI}(n - 1, \text{tmp}, \text{dst}, \text{src})$ 

```

Initial call: $\text{HANOI}(n, 1, 3, 2)$

Trace for $n = 3$

Move	From $\rightarrow$ To
1	$1 \rightarrow 3$
2	$1 \rightarrow 2$
3	$3 \rightarrow 2$
4	$1 \rightarrow 3$ (largest disk)
5	$2 \rightarrow 1$
6	$2 \rightarrow 3$
7	$1 \rightarrow 3$

اگر $T(n)$ تعداد حرکات‌ها باشد:

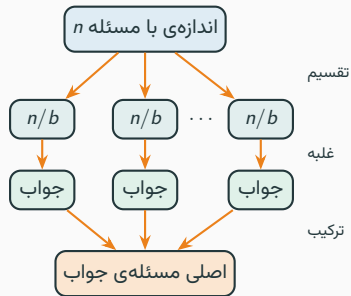
$$T(n) = \begin{cases} 1 & n = 1 \\ 2T(n-1) + 1 & n > 1 \end{cases}$$

باز کردن رابطه (جای‌گذاری با تکرار):

$$\begin{aligned} T(n) &= 2T(n-1) + 1 \\ &= 4T(n-2) + 2 + 1 \\ &= 2^k T(n-k) + (2^k - 1) \\ &= 2^{n-1} + 2^{n-1} - 1 \quad (k = n-1) \\ &= 2^n - 1 = \Theta(2^n) \end{aligned}$$

مقدمه‌ای بر روش تقسیم و حل

الگوی تقسیم و غلبه (Divide and Conquer)



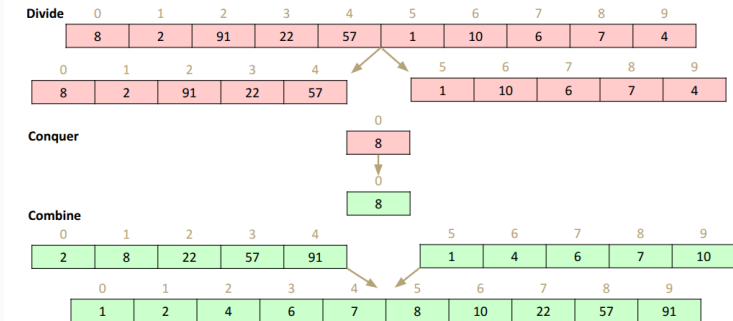
۱. **تقسیم** (divide): مسئله را به چند زیرمسئله از همان نوع و با اندازه‌ی کوچکتر بشکن.

۲. **حل/غلبه** (conquer): زیرمسئله‌ها را به صورت بازگشتی حل کن.

۳. **ترکیب** (combine): از جواب زیرمسئله‌ها جواب مسئله‌ی اصلی را بساز.

بر اساس CLRS، صفحه ۷۶

Merge Sort



CSE 373 23SP

29

UW CSE 373 23SP, Lecture 6, slide 29 — [Link](#)

Merge Sort

```

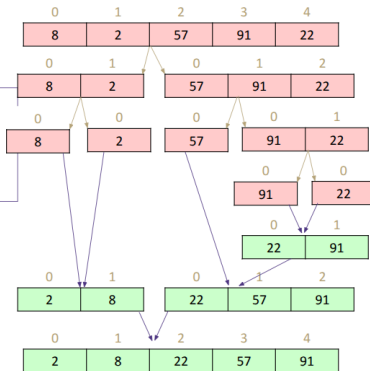
mergeSort(input) {
  if (input.length == 1)
    return
  else
    smallerHalf = mergeSort(new [0, ..., mid])
    largerHalf = mergeSort(new [mid + 1, ...])
    return merge(smallerHalf, largerHalf)
}

```

$$T(n) = \begin{cases} 1 & \text{if } n \leq 1 \\ 2T(n/2) + n & \text{otherwise} \end{cases}$$

Pattern #2 – Constant size input and doing work

Take a guess! What is the Big-O of worst case merge sort?



Merge Sort Recurrence to Big-Θ

$$T(n) = \begin{cases} 1 & \text{if } n \leq 1 \\ 2T(n/2) + n & \text{otherwise} \end{cases}$$

Master Theorem

$$T(n) = \begin{cases} d & \text{if } n \text{ is at most some constant} \\ aT\left(\frac{n}{b}\right) + f(n) & \text{otherwise} \end{cases}$$

Where $f(n)$ is $\Theta(n^c)$

- If $\log_b a < c$ then $T(n) \in \Theta(n^c)$
- If $\log_b a = c$ then $T(n) \in \Theta(n^c \log n)$
- If $\log_b a > c$ then $T(n) \in \Theta(n^{\log_b a})$



$a=2$ $b=2$ and $c=1$

$y = \log_b x$ is equal to $b^y = x$

$$\log_2 2 = x \Rightarrow 2^x = 2 \Rightarrow x = 1$$

$$\log_2 2 = 1$$

We're in case 2

$$T(n) \in \Theta(n \log n)$$

منابع

- داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، انتشارات فاطمی
- Cormen, Leiserson, Rivest, Stein — *Introduction to Algorithms* (CLRS), 4th Edition
- Jeff Erickson — *Algorithms*
- University of Washington CSE 373 23SP - Section 2
- University of Washington CSE 373 23SP - Lecture 6: Analyzing Recursive Code